

hdlo_models.cpp Auto-Generation Format

File Structure

```
cpp

//////////
// HDLO Models

// define the data:

#include "hdlo_models.h"

// =====
// SECTION 1: MODEL DATA ARRAYS
// =====
// Format: std::array<std::array<int, 6>, 120> <modelname>data={{...}};
// Each model has 120 entries (edges)
// Each entry is an array of 6 integers: [color_func_index, ?, start_pos, end_pos, ?, ?]

std::array<std::array<int, 6>, 120> flowoctahedrondata={{
    {{0,1,0,10000,0,10000}},
    {{0,1,0,10000,0,10000}},
    // ... 120 total entries
    {{0,0,0,0,0,0}}, // Unused entries are zeroed
}};

std::array<std::array<int, 6>, 120> flowoctahedroncompound1data={{
    // ... 120 entries
}};

// Repeat for each model...

// =====
// SECTION 2: MODEL METADATA
// =====

colormodel* ourcolormodels[N]; // N = number of models

// Model names array - MUST match the order in ourcolormodels
String modelNames[N] = {
    "flowoctahedron",
    "flowoctahedroncompound1",
    "octachainshiftedby",
    // ... one name per model
};

// =====
// SECTION 3: MODEL INITIALIZATION
// =====

void initializemodels(){
    ourcolormodels[0] = new colormodel(flowoctahedrondata, modelNames[0]);
    ourcolormodels[1] = new colormodel(flowoctahedroncompound1data, modelNames[1]);
    ourcolormodels[2] = new colormodel(octachainshiftedbydata, modelNames[2]);
    // ... one line per model using pattern:
    // ourcolormodels[i] = new colormodel(<modelname>data, modelNames[i]);
}
```

Generation Rules

Section 1: Model Data Arrays

- **Variable naming:** `<modelname>data` where modelname is lowercase with no spaces
- **Type:** Always `std::array<std::array<int, 6>, 120>`

- **Size:** Always 120 entries (one per edge)
- **Entry format:** `{{val0, val1, val2, val3, val4, val5}}`
- **Unused entries:** Fill with `{{0,0,0,0,0,0}}`

Section 2: Model Metadata

- **Array size:** Must match number of models
- **modelName order:** MUST match the order models are created in `initializemodels()`
- **Naming convention:** Lowercase, no spaces (e.g., "flowoctahedron", "cyclestemplate")

Section 3: Model Initialization

- **Pattern:** `ourcolormodels[i] = new colormodel(<modelname>data, modelNames[i]);`
- **Order:** Must match modelNames array order
- **Index:** Start at 0, increment by 1

Complete Example

```
cpp

////////////////////
// HDLO Models

#include "hdlo_models.h"

// Model data arrays
std::array<std::array<int, 6>, 120> exampledata={{
    {{0,1,0,10000,0,10000}},
    {{1,1,5000,5000,0,10000}},
    {{2,1,2500,2500,0,10000}},
    {{0,0,0,0,0,0}}, // Entry 3
    // ... entries 4-119 all zeros
}};

std::array<std::array<int, 6>, 120> anotherdata={{
    // ... 120 entries
}};

std::array<std::array<int, 6>, 120> thirddata={{
    // ... 120 entries
}};

// Model metadata
colormodel* ourcolormodels[3];

String modelNames[3] = {
    "example",
    "another",
    "third"
};

// Model initialization
void initializemodels(){
    ourcolormodels[0] = new colormodel(exampledata, modelNames[0]);
    ourcolormodels[1] = new colormodel(anotherdata, modelNames[1]);
    ourcolormodels[2] = new colormodel(thirddata, modelNames[2]);
}
```

Constants and Constraints

- **Maximum models:** Configurable via `nummodels` constant in `hdlo_models.h`
- **Edges per model:** Always 120
- **Values per edge:** Always 6 integers
- **Model naming:** Lowercase, no spaces or special characters

Key Points for Auto-Generation

1. **Consistency is critical:** modelNames[i] must match the i-th model created
2. **All names lowercase:** Model names should be lowercase with no spaces
3. **Data variable naming:** Always `<modelName>data`
4. **Array size declarations:** Update both the extern declarations in .h and the array sizes in .cpp

Files That Reference hdlo_models

The following files use data from hdlo_models:

- **sequences.cpp** - Uses `modelNames[]` array to reference models
- **hdlo_controller.ino** - Uses `ourcolormodels[]` and `modelNames[]` arrays
- **modelsequence.cpp** - Generic, no model-specific references

What Gets Auto-Generated

Only these files need auto-generation:

1. **hdlo_models.h** - Update `nummodels` constant and array sizes
2. **hdlo_models.cpp** - Complete file (all 3 sections)

Do NOT auto-generate:

- sequences.h
- sequences.cpp (hand-coded choreography)

hdlo_models.h Format

```
cpp

////////////////////////////////////
// HDLO Models

#ifndef HDLO_MODELS_H
#define HDLO_MODELS_H

#include <array>
#include "models.h"

// Declare all model data arrays
extern std::array<std::array<int, 6>, 120> flowoctahedrondata;
extern std::array<std::array<int, 6>, 120> flowoctahedroncompound1data;
extern std::array<std::array<int, 6>, 120> octachainshiftedbydata;
// ... one declaration per model

// Model count and arrays
const int nummodels = 6; // UPDATE THIS
extern colormodel* ourcolormodels[6]; // UPDATE SIZE
extern String modelNames[6]; // UPDATE SIZE

void initializemodels();

#endif // HDLO_MODELS_H
```

Summary

Auto-generate these files:

- hdlo_models.h (declarations + array sizes)
- hdlo_models.cpp (data arrays + metadata + initialization)

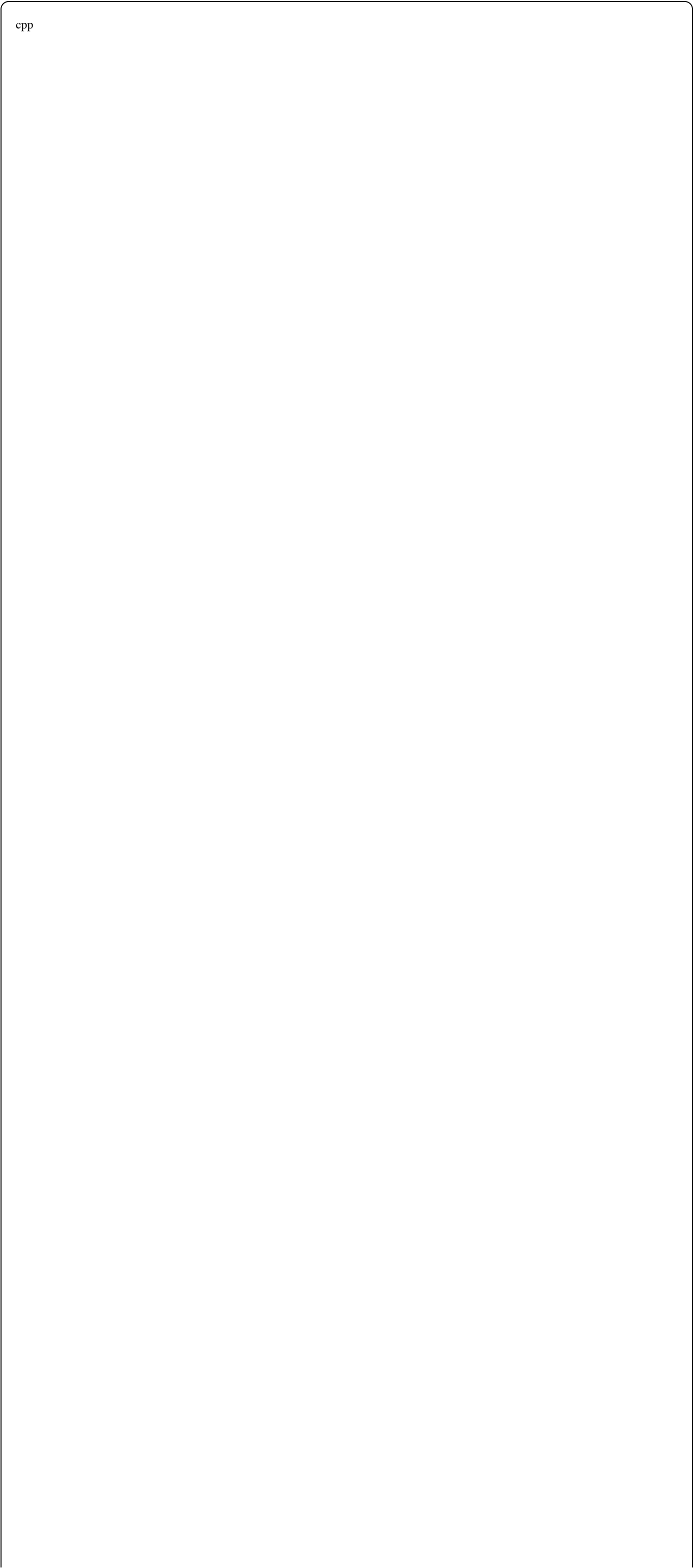
Hand-code these files:

- sequences.h (simple header)
- sequences.cpp (choreography using `modelNames[]`)

No changes needed:

- All other files work generically# hdlo_models.cpp Auto-Generation Format

File Structure



```
//////////
// HDLO Models

// define the data:

#include "hdlo_models.h"

// =====
// SECTION 1: MODEL DATA ARRAYS
// =====
// Format: std::array<std::array<int, 6>, 120> <modelname>data={{...}};
// Each model has 120 entries (edges)
// Each entry is an array of 6 integers: [color_func_index, ?, start_pos, end_pos, ?, ?]

std::array<std::array<int, 6>, 120> flowoctahedrondata={{
    {{0,1,0,10000,0,10000}},
    {{0,1,0,10000,0,10000}},
    // ... 120 total entries
    {{0,0,0,0,0,0}}, // Unused entries are zeroed
}};

std::array<std::array<int, 6>, 120> flowoctahedroncompound1data={{
    // ... 120 entries
}};

// Repeat for each model...

// =====
// SECTION 2: MODEL METADATA
// =====

colormodel* ourcolormodels[N]; // N = number of models

// Model names array - MUST match the order in ourcolormodels
String modelNames[N] = {
    "flowoctahedron",
    "flowoctahedroncompound1",
    "octachainshiftedby",
    // ... one name per model
};

// =====
// SECTION 3: MODEL INITIALIZATION
// =====

void initializemodels(){
    ourcolormodels[0] = new colormodel(flowoctahedrondata, modelNames[0]);
    ourcolormodels[1] = new colormodel(flowoctahedroncompound1data, modelNames[1]);
    ourcolormodels[2] = new colormodel(octachainshiftedbydata, modelNames[2]);
    // ... one line per model using pattern:
    // ourcolormodels[i] = new colormodel(<modelname>data, modelNames[i]);
}

// =====
// SECTION 4: SEQUENCE DEFINITIONS
// =====

#include "modelsequence.h"

void initializeSequences(modelsequence* seq) {
    // Clear any existing steps
    seq->clearRegistry();

    // Pattern for each sequence:
    // 1. Start new sequence with name and duration
```

```
seq->startNewSequence("Sequence Name", duration_ms, enabled);

// 2. Add steps using model indices and color function names
seq->addStepByName(modelNames[index],
    {"color1", "color2", "color3", "color4"},
    step_duration_ms,
    TRANSITION_TYPE, // INSTANT, FADE, or WIPE
    transition_duration_ms);

// Repeat addStepByName for each step in the sequence

// 3. Start next sequence
seq->startNewSequence("Next Sequence", duration_ms, enabled);
// ... more steps

// Example of a complete sequence:
seq->startNewSequence("Default Flow", 45000, true);
seq->addStepByName(modelNames[0], {"dark", "rainbow", "bluetored", "staticblue"}, 8000, FADE, 1000);
seq->addStepByName(modelNames[1], {"dark", "bluetored", "rainbow", "staticblue"}, 6000, WIPE, 1500);
seq->addStepByName(modelNames[2], {"dark", "staticblue", "bluetored", "rainbow"}, 7000, INSTANT, 0);
// ... more steps
}
```

Generation Rules

Section 1: Model Data Arrays

- **Variable naming:** `<modelname>data` where modelname is lowercase with no spaces
- **Type:** Always `std::array<std::array<int, 6>, 120>`
- **Size:** Always 120 entries (one per edge)
- **Entry format:** `{{val0, val1, val2, val3, val4, val5}}`
- **Unused entries:** Fill with `{{0,0,0,0,0,0}}`

Section 2: Model Metadata

- **Array size:** Must match number of models
- **modelNames order:** MUST match the order models are created in `initializemodels()`
- **Naming convention:** Lowercase, no spaces (e.g., "flowoctahedron", "cyclestemplate")

Section 3: Model Initialization

- **Pattern:** `ourcolormodels[i] = new colormodel(<modelname>data, modelNames[i]);`
- **Order:** Must match modelNames array order
- **Index:** Start at 0, increment by 1

Section 4: Sequence Definitions

- **Sequence structure:**
 1. Call `startNewSequence(name, duration_ms, enabled)`
 2. Add N steps with `addStepByName()`
 3. Repeat for next sequence
- **addStepByName parameters:**
 - `modelNames[index]`: Reference to model by index
 - `{"color1", "color2", "color3", "color4"}`: Always 4 color function names
 - Available color names: "dark", "rainbow", "bluetored", "staticblue"
 - `step_duration_ms`: How long this step lasts
 - `TRANSITION_TYPE`: INSTANT, FADE, or WIPE
 - `transition_duration_ms`: How long transition takes (0 for INSTANT)
- **Sequence timing:**
 - `duration_ms` in `startNewSequence`: Total time to display this entire sequence
 - `step_duration_ms` in `addStepByName`: Time for individual step within sequence

Complete Example

```
cpp

// Model with 3 entries (rest would be zeros)
std::array<std::array<int, 6>, 120> exampledata={{
    {{0,1,0,10000,0,10000}},
    {{1,1,5000,5000,0,10000}},
    {{2,1,2500,2500,0,10000}},
    {{0,0,0,0,0,0}}, // Entry 3
    // ... entries 4-119 all zeros
}};

String modelNames[3] = {
    "example",
    "another",
    "third"
};

void initializeModels(){
    ourcolormodels[0] = new colorModel(exampledata, modelNames[0]);
    ourcolormodels[1] = new colorModel(anotherdata, modelNames[1]);
    ourcolormodels[2] = new colorModel(thirddata, modelNames[2]);
}

void initializeSequences(modelSequence* seq) {
    seq->clearRegistry();

    seq->startNewSequence("Fast Cycle", 30000, true);
    seq->addStepByName(modelNames[0], {"dark", "rainbow", "rainbow", "rainbow"}, 5000, FADE, 1000);
    seq->addStepByName(modelNames[1], {"dark", "bluetored", "staticblue", "rainbow"}, 5000, WIPE, 500);

    seq->startNewSequence("Slow Flow", 60000, true);
    seq->addStepByName(modelNames[2], {"dark", "staticblue", "bluetored", "staticblue"}, 15000, FADE, 3000);
    seq->addStepByName(modelNames[0], {"dark", "rainbow", "bluetored", "staticblue"}, 15000, INSTANT, 0);
}
```

Constants and Constraints

- **Maximum models:** Limited by `nummodels` constant in `hdlo_models.h`
- **Edges per model:** Always 120
- **Values per edge:** Always 6 integers
- **Color functions:** "dark", "rainbow", "bluetored", "staticblue" (4 always needed per step)
- **Maximum sequences:** 10 (`MAX_REGISTRY_ENTRIES`)
- **Maximum steps per sequence:** 20 total across all sequences (`MAX_STEPS`)
- **Transition types:** INSTANT, FADE, WIPE

Key Points for Auto-Generation

1. **Consistency is critical:** `modelNames[i]` must match the i-th model created
2. **All names lowercase:** Model names should be lowercase with no spaces
3. **Data variable naming:** Always `<modelname>data`
4. **Always 4 colors:** Each step needs exactly 4 color function names
5. **Sequence order:** Sequences are played in the order defined
6. **Step accumulation:** Steps accumulate across sequences (track total < 20)